

CS4031 Assignment 03
Bottom-Up Parser Design & Implementation
SLR(1) and LR(1) Parsing

Umar Javed (22i-1050) Sheharyar Ahmed (22i-1171)

Spring 2026

Course: Compiler Construction

Language Used: Java

Contents

1	Introduction	3
2	Approach	3
2.1	Data Structures	3
2.2	Grammar Handling	3
2.3	LR(0) Item Construction	4
2.4	LR(1) Item Construction	4
2.5	Parsing Table Construction	4
2.5.1	SLR(1)	4
2.5.2	LR(1)	4
2.6	Parse Tree Generation	4
2.7	Design Decisions and Trade-Offs	5
3	Challenges	5
4	Test Cases	5
4.1	Simple Expression Grammar	5
4.2	Expression Grammar With Precedence	5
4.3	Classic SLR(1) Conflict Grammar	6
4.4	Dangling Else Grammar	6
4.5	Input Sets	6
4.6	Sample Inputs Used	6
5	Comparison Analysis	7
5.1	SLR(1) vs LR(1) Parsing Power	7
5.2	State Count and Table Size	7
5.3	Construction Time	7
5.4	Parsing Speed	7
5.5	Conflict Behavior	7
6	Sample Outputs	7
6.1	SLR Trace Example	8
6.2	LR(1) Conflict Resolution Example	8
6.3	Parse Tree Example	8
6.4	Screenshots	8
7	Conclusion	9
	Appendix: Folder Structure	9

1 Introduction

Bottom-up parsing constructs a parse tree by repeatedly reducing input symbols to higher-level grammar symbols until the start symbol is reached. Unlike top-down parsing, which predicts productions from the start symbol downward, bottom-up parsing works from the input string upward and is the basis of LR-family parsers.

This assignment implements two bottom-up parser variants:

- SLR(1), which uses LR(0) item sets and FOLLOW sets for reduce actions.
- LR(1), which uses LR(1) items with explicit lookahead symbols for more precise reductions.

The implementation reads a context-free grammar from a text file, augments it with a new start symbol, constructs canonical item collections, builds parsing tables, parses test inputs using shift-reduce parsing, detects conflicts, and generates parse trees for accepted strings.

2 Approach

2.1 Data Structures

The parser is organized into reusable Java classes:

- **Grammar** stores productions, terminal and non-terminal sets, FIRST sets, FOLLOW sets, and grammar augmentation logic.
- **Items** defines LR(0) and LR(1) item representations and builds canonical automata.
- **ParsingTable** stores ACTION and GOTO entries and records conflicts.
- **Stack** implements the parser stack used during shift-reduce parsing.
- **Tree** and **TreeNode** represent parse trees.
- **SLRParser** and **LR1Parser** build parser tables and execute parsing.
- **Main** coordinates file input, parser execution, and output generation.

2.2 Grammar Handling

The grammar file uses one production per line in the form:

<code>NonTerminal -> production1 production2 production3</code>
--

The parser supports multi-character non-terminals, terminals written as tokens, and epsilon productions using either `epsilon` or `@`. The grammar is augmented by adding a new start symbol of the form `StartPrime -> Start`.

2.3 LR(0) Item Construction

For SLR(1), canonical LR(0) item sets are built using the standard closure and goto operations:

- `closure(I)` adds items for non-terminals appearing immediately after the dot.
- `goto(I, X)` moves the dot over grammar symbol `X` and then applies closure.

The canonical collection is built by repeatedly applying goto to each state and each grammar symbol until no new item sets are discovered.

2.4 LR(1) Item Construction

LR(1) items extend LR(0) items by adding a lookahead terminal. The closure operation propagates lookahead information using FIRST sets of the suffix after the non-terminal plus the current lookahead. This makes LR(1) reductions more precise than SLR(1).

2.5 Parsing Table Construction

2.5.1 SLR(1)

- Shift actions are added when an item has a terminal immediately after the dot.
- Reduce actions are added for completed items using FOLLOW(lhs).
- Accept is added when the augmented start item completes on \$.
- GOTO entries are added for non-terminals.

2.5.2 LR(1)

- Shift actions are added the same way as SLR(1).
- Reduce actions are added only for the specific lookahead in the LR(1) item.
- Accept is added for [StartPrime -> Start ., \$].
- GOTO entries are added the same way as SLR(1).

2.6 Parse Tree Generation

Each reduction creates a parse tree node whose children are the symbols reduced from the stack. When parsing succeeds, the final tree root is the original start symbol. The tree is written in a readable indented form.

2.7 Design Decisions and Trade-Offs

The project uses a modular structure so that grammar handling, item-set construction, table construction, and parsing are separated. This makes the SLR(1) and LR(1) implementations share the same grammar and stack infrastructure while differing only in item-set logic and reduce lookahead rules.

A deliberate trade-off was to keep the parser token-based and avoid lexical analysis. This keeps the implementation focused on parser construction, which matches the assignment requirements.

3 Challenges

1. Correctly handling epsilon in FIRST computations and LR(1) closure propagation.
2. Ensuring grammar augmentation happens before item-set construction.
3. Detecting shift/reduce and reduce/reduce conflicts without overwriting earlier table decisions silently.
4. Generating parse trees that reflect the actual reduction sequence.
5. Keeping the implementation generic enough to work across multiple grammars while still remaining simple to test.

The most important fix was handling LR(1) lookahead propagation carefully so that reductions occur only when the item's specific lookahead matches the current input symbol.

4 Test Cases

The following grammars were added under `input/`.

4.1 Simple Expression Grammar

`grammar1.txt`

```
Expr -> Expr + Term | Term
Term -> Factor
Factor -> id
```

4.2 Expression Grammar With Precedence

`grammar2.txt`

```
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id
```

4.3 Classic SLR(1) Conflict Grammar

grammar3.txt

```
Start -> L = R | R
L -> * R | id
R -> L
```

This grammar is a standard example where SLR(1) has a conflict while LR(1) resolves it.

4.4 Dangling Else Grammar

grammar4.txt

```
Stmt -> if Expr then Stmt | if Expr then Stmt else Stmt | other
Expr -> id
```

4.5 Input Sets

The following test input files are included:

- *grammar1_iinputs.txt*
- *grammar2_iinputs.txt*
- *grammar3_iinputs.txt*
- *grammar4_iinputs.txt*
- *input_vvalid.txt*
- *input_invalid.txt*

These include valid strings, invalid strings, empty input handling, nested structures, and conflict-triggering strings.

4.6 Sample Inputs Used

Examples include:

- `id + id * id`
- `( id + id ) * id`
- `id = id`
- `if id then other else other`
- invalid strings such as `id + + id` and `( id + id`

5 Comparison Analysis

5.1 SLR(1) vs LR(1) Parsing Power

SLR(1) uses FOLLOW sets, so its reduce actions are less precise. LR(1) uses item-specific lookaheads and can resolve conflicts that SLR(1) cannot.

The grammar in `grammar3.txt` demonstrates this difference. In the generated table output, SLR reports a conflict while LR(1) does not.

5.2 State Count and Table Size

A sample run on `grammar2.txt` produced:

Metric	SLR(1)	LR(1)
Number of states	12	22
Table entries	45	71

This shows the usual trade-off: LR(1) is more powerful but often larger.

5.3 Construction Time

A sample run on `grammar2.txt` showed that both parsers complete table construction quickly on assignment-sized grammars. The exact values vary by grammar and machine, but the build times remain practical for classroom-scale inputs.

5.4 Parsing Speed

Parsing speed is also fast for the provided sample inputs. The comparison output written by the program records parse time for each test string in both parsers.

5.5 Conflict Behavior

For `grammar3.txt`:

- SLR(1) reports a shift/reduce conflict.
- LR(1) resolves the conflict and produces a conflict-free table.

For `grammar4.txt`, the implementation also exposes the kind of ambiguity that must be reported during table construction.

6 Sample Outputs

The program writes all major artifacts into `output/`:

- `augmented_grammar.txt`

- `slr_items.txt`
- `slr_parsing_table.txt`
- `slr_trace.txt`
- `lr1_items.txt`
- `lr1_parsing_table.txt`
- `lr1_trace.txt`
- `comparison.txt`
- `parse_trees.txt`

6.1 SLR Trace Example

A successful run for the expression grammar shows a shift-reduce sequence ending in accept.

6.2 LR(1) Conflict Resolution Example

For the classic conflict grammar, the SLR table includes a conflict, while the LR(1) table is conflict-free.

6.3 Parse Tree Example

For `id + id * id`, the parse tree is generated from the reduction sequence and shows the correct hierarchy of expression, term, factor, and terminal nodes.

6.4 Screenshots

Insert screenshots here from the generated output files or terminal execution:

- canonical item sets
- parsing tables
- parsing traces
- parse trees
- conflict messages

7 Conclusion

This assignment demonstrates how bottom-up parsing works in practice and highlights the difference between SLR(1) and LR(1) parsing power. The SLR(1) parser is simpler and smaller, but it depends on FOLLOW sets and can produce conflicts for grammars that LR(1) handles correctly. LR(1) is more precise because it carries lookahead information directly in items.

The implementation is modular, reusable, and organized for testing across multiple grammars. It also generates parse trees and trace outputs, which makes the parser behavior easy to inspect and verify.

Appendix: Folder Structure

```
Assignment 03/
+-- src/
| +-- Main.java
| +-- Grammar.java
| +-- SLRParser.java
| +-- LR1Parser.java
| +-- Items.java
| +-- ParsingTable.java
| +-- Stack.java
| +-- Tree.java
+-- input/
| +-- grammar1.txt
| +-- grammar1_inputs.txt
| +-- grammar2.txt
| +-- grammar2_inputs.txt
| +-- grammar3.txt
| +-- grammar3_inputs.txt
| +-- grammar4.txt
| +-- grammar4_inputs.txt
| +-- grammar_with_conflict.txt
| +-- input_valid.txt
| +-- input_invalid.txt
+-- output/
+-- docs/
| +-- report.tex
+-- build.bat
+-- README.md
```